

Indian Institute of Technology Madras
Department of Data Science and Artificial Intelligence

DA2304: Introduction to Computer Systems

Assignment 2: Virtual Machines & Matrix Operations

Name: *Mohammed Khaja Ghouse Mohiuddin*
Roll No.: *DA24B015*
Instructor: Dr. Patanjali SLPSK
Deadline: April 30, 2026

1 Part 1: VM Translator ; Implementation Overview

1.1 Module Architecture

`parser.py`

Designed to remove inline comments (`//`), tokenise each line, and classify it into one of nine command types: `C_ARITHMETIC`, `C_PUSH`, `C_POP`, `C_LABEL`, `C_GOTO`, `C_IF`, `C_FUNCTION`, `C_CALL`, and `C_RETURN`. It also provides `arg1()` and `arg2()` accessor methods.

`code_writer.py`

Receives the classified commands and generates Hack assembly code. A monotonically increasing label counter is used to ensure unique jump labels for comparison operations across multiple translated files. `R13` is used as a scratch register for computing target addresses in `pop` commands, while `R14` and `R15` store the frame pointer and return address during `return` execution.

`main.py`

Acts as the entry point. It accepts either a single `.vm` file or a directory. For directory input, it first emits bootstrap code (`SP = 256`, `call Sys.init`) and then translates each file in sorted order, producing a single combined `.asm` output.

1.2 Supported Commands

Category	Commands
Arithmetic / Logic	<code>add</code> <code>sub</code> <code>neg</code> <code>eq</code> <code>gt</code> <code>lt</code> <code>and</code> <code>or</code> <code>not</code>
Memory Access	<code>push</code> / <code>pop</code> ; <code>constant</code> <code>local</code> <code>argument</code> <code>this</code> <code>that</code> <code>static</code> <code>temp</code> <code>pointer</code>
Program Flow	<code>label</code> <code>goto</code> <code>if-goto</code>
Functions	<code>function</code> <code>call</code> <code>return</code>

1.3 Memory Segment Mapping

VM Segment	Hack RAM mapping
constant	Literal immediate value (push only)
local	RAM[LCL + i]
argument	RAM[ARG + i]
this	RAM[THIS + i]
that	RAM[THAT + i]
static	RAM[filename.i] (assembled as a named symbol)
temp	RAM[5 + i] ($i \in \{0, \dots, 7\}$)
pointer	THIS ($i = 0$) or THAT ($i = 1$)

2 Part 2: Matrix MAC Operations

We know that

$$Y = W \cdot X + B$$

where $W \in \mathbb{Z}^{m \times k}$, $X \in \mathbb{Z}^{k \times n}$, and $B \in \mathbb{Z}^{m \times 1}$ is a bias column vector.

2.1 Exercise 1.1 ; Sanity Check

since hack VM has no native exception mechanism, so we can make errors signalled by writing -1 to a reserved memory cell ($\text{RAM}[\text{FLAG}]$) and then jumping to an infinite halt loop. On success, we can set $\text{RAM}[\text{FLAG}]$ to 1 .

```

1 //W is (m x k), X is (k x n), B is (m x 1)
2 //Stored metadata: RAM[W_ROWS]=m, RAM[W_COLS]=k,
3 //RAM[X_ROWS]=k, RAM[X_COLS]=n, RAM[B_ROWS]=m
4
5 FUNCTION sanity_check():
6     IF W_COLS != X_ROWS:
7         RAM[FLAG] = -1           //dimension mismatch W*X
8         GOTO ERROR             //infinite halt loop
9     IF W_ROWS != B_ROWS:
10        RAM[FLAG] = -1          //dimension mismatch +B
11        GOTO ERROR
12    RAM[FLAG] = 1               //all checks passed
13    RETURN
14
15 label ERROR
16    goto ERROR                 //trap ; visible in emulator memory

```

Listing 1: Dimension sanity check pseudocode

Implementation Note: In the actual VM code, `GOTO ERROR` is implemented using an `if-goto` after pushing the comparison result onto the stack. Setting $\text{RAM}[\text{FLAG}] = -1$ acts as a clear indicator (sentinel) when inspecting memory in the CPU Emulator after a failed run.

2.2 Exercise 1.2 ; Approach A: MATMUL Function Call

A single function $\text{MATMUL}(W_ptr, X_ptr, m, k, n)$ is called once. It owns the complete triple-nested loop internally and returns a pointer to the output matrix Y .

```

1 FUNCTION MATMUL(W_ptr, X_ptr, m, k, n):
2     allocate Y[m][n] starting at Y_ptr
3     FOR i = 0 TO m-1:
4         FOR j = 0 TO n-1:
5             acc = 0
6             FOR p = 0 TO k-1:
7                 acc += RAM[W_ptr + i*k + p] * RAM[X_ptr + p*
8                     n + j]
9                 RAM[Y_ptr + i*n + j] = acc + RAM[B_ptr + i]
10    RETURN Y_ptr

```

Listing 2: Approach A ; MATMUL pseudocode

2.3 Exercise 1.3 ; Approach B: ROWXCOL Subroutine

The will manage the outer two loops and calls ROWXCOL(row_ptr, col_ptr, length) once per output element. The subroutine returns the scalar dot product Y_{ij} ie., the caller adds the bias afterwards.

```
1 //caller:
2 FOR i = 0 TO m-1:
3     FOR j = 0 TO n-1:
4         row_ptr = W_ptr + i * k          // pointer to row i of
           W
5         col_ptr = X_ptr + j             // pointer to col j of
           X
6         Y[i][j] = ROWXCOL(row_ptr, col_ptr, k) + B[i]
7
8 //subroutine:
9 FUNCTION ROWXCOL(row_ptr, col_ptr, length):
10     acc = 0
11     FOR p = 0 TO length-1:
12         acc += RAM[row_ptr + p] * RAM[col_ptr + p * n]
13     RETURN acc
```

Listing 3: Approach B ; ROWXCOL pseudocode

3 Part 2: Exercise 2 ; Architectural Analysis

3.1 Q1 ; Stack Usage and Function-Call Overhead

3.1.1 Approach A (MATMUL)

MATMUL is invoked **once**. A single call frame is pushed, which is *return address* + saved LCL, ARG, THIS, THAT, which is equal to **5 words** of overhead. on top of this the the function's declared locals (like loop counters, accumulator). All arithmetic happens inside this single frame. ie., stack depth remains essentially constant throughout the $m \cdot k \cdot n$ inner iterations.

3.1.2 Approach B (ROWXCOL)

ROWXCOL is called $m \times n$ times, which is once per output element. Each call pushes the same 5-word call frame and then executes k multiply-add iterations and then tears the frame down using **return**. Although the instantaneous stack depth will never exceed one call frame, the total push and pop traffic is $\mathcal{O}(m \cdot n)$ times larger than Approach A. For a 512×512 matrix this means 262,144 separate call/return sequences, which dominates execution time.

3.2 Q2 ; Memory Segment Utilisation

Segment	Approach A	Approach B
local	Heavy use; i, j, p loop counters and the running accumulator all reside here for the lifetime of the single call frame.	Caller holds i, j in its local . ROWXCOL's frame holds only p and acc , but these are allocated and freed $m \cdot n$ times.
argument	set once: W_ptr , X_ptr , m , k , n . Never touched again inside the call.	re-pushed on every one of the $m \cdot n$ calls: row_ptr , col_ptr , length .
temp	Scratch for address arithmetic. Available without frame overhead.	Scratch for stride calculation ($p \cdot n$ in col-pointer advance).
static	Bias vector B which is globally accessible by label, no pointer arithmetic needed.	Same.
this / that, pointer	not used.	not used.

The main difference is that in Approach A, the **local** segment is allocated **only once** and reused across all $m \cdot k \cdot n$ iterations. On the other hand, Approach B repeatedly allocates and deallocates the callee's **local** and **argument** segments for each of the $m \cdot n$ function calls, which increases the number of segment-setup instructions proportionally.

3.3 Q3: Optimisations for Approach A (MATMUL)

3.3.1 Software-level

- **Loop unrolling.** we can expand the innermost accumulation loop by a factor of 4 or 8. Each unrolled iteration eliminates one branch instruction and one loop-counter

decrement, cutting branch overhead by up to $8\times$ at the cost of larger code size.

- **Accumulator in temp.** Keep the running sum in `temp 0` (i.e. `RAM[5]`) instead of the `local` segment. Accessing `temp` requires only an `@5 A`-instruction, whereas `local` requires a base-pointer dereference through `LCL`; saving 1–2 instructions per inner iteration.
- **Strength reduction on address arithmetic.** Replace $i \cdot k + p$ multiplication (itself an $\mathcal{O}(k)$ loop on Hack) with an incrementing pointer: initialise `w_ptr = W_ptr + i*k` once per row, then increment by 1 per inner step. This eliminates $m \cdot n \cdot k$ multiply calls.
- **Cache tiling.** Divide W and X into sub-blocks that fit within the 8 available `temp` registers (`R5–R12`). Processing one tile at a time maximises register reuse and minimises repeated RAM reads for the same matrix elements.

3.3.2 Hardware-level

- **Dedicated MUL instruction.** The base Hack CPU has no multiply opcode. each integer multiplication is a software loop of $\mathcal{O}(b)$ additions where b is the operand. Adding a single-cycle hardware multiplier collapses the inner loop's dominant cost to one instruction.
- **SIMD / vector ALU.** A vector unit processing 4 or 8 words per cycle reduces the inner dot-product from k cycles to $\lceil k/8 \rceil$ cycles.
- **Wider data bus.** Widening the Hack bus from 16 bits to 32 or 64 bits allows two or four matrix elements to be fetched per cycle, halving or quartering the memory latency that dominates the access-heavy inner loop.

3.4 Q4 ; Compiler-level Optimisations for Approach B (ROWXCOL)

- **Function inlining.** we can make the compiler replace every `call ROWXCOL` with the body of `ROWXCOL` directly in the caller, eliminating all $m \cdot n$ call frames (5 pushes + 5 frame restores + 2 unconditional jumps each). For like small and fixed-size matrices this is like a win-win
- **Passing arguments using temp.** If the compiler can ensure that `ROWXCOL` does not overwrite certain `temp` registers, it can pass `row_ptr`, `col_ptr`, and `length` through `temp` instead of using the formal `argument` segment, thereby avoiding the need to set up the `ARG` pointer altogether.
- **Partial evaluation / full unrolling.** When `length` (i.e. k) is known at compile time, the compiler can completely unroll the inner loop of `ROWXCOL`, generating a straight sequence of k multiply-add operations without any branching.
- **Loop-invariant code motion.** In the given code, the caller recomputes `row_ptr = W_ptr + i*k` for every j inside the inner loop. Since this value does not change with j , the compiler can identify it as loop-invariant and move it outside the j -loop. This avoids redundant work, saving $n - 1$ multiply subroutine calls per row, effectively reducing that computation by a factor of n .
- **Tail-call / return elimination.** After `ROWXCOL` returns, the caller performs a trivial add-bias-and-store. Restructuring so the callee writes its result directly to

$Y[i][j]$ before returning enables the compiler to merge the return with the store, removing one stack-top read and one store instruction.

Extra Credit

A Thousand Years by John Michael Howell(ft. JVKE, ZVC) is my favourite song, recently
